

**UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS**

**MANUAL DE INSTRUÇÃO PARA CONSTRUÇÃO DE MODELOS URDF
GUIA PRÁTICO DE MODELAGEM E ESTRUTURAÇÃO NO ROS 2**

FELIPE PEREIRA FURLANETO

ffurlaneto@usp.br

Manual de instruções técnicas focado no desenvolvimento, parametrização e validação de arquivos URDF para simulação cinemática e dinâmica de robôs.

**SÃO CARLOS
2026**

Sumário

1	Preparação das Partes e Malhas Tridimensionais	2
1.1	Organização Estrutural e Preparação de Malhas no Blender	3
1.1.1	Passo 1: Importação e Configuração do Ambiente	3
1.1.2	Passo 2: O Desafio da Escala	5
1.1.3	Passo 3: Correção do Centro Geométrico e Pontos de Articulação (Origem)	5
1.1.4	Passo 4: Junção de Peças Estáticas (<i>Join</i>)	8
1.1.5	Passo 5: Exportação para COLLADA (.dae)	10
1.1.6	Importação, Otimização e Customização de Arquivos STL	12
2	Criação da Pasta, montagem do URDF e configuração do ambiente	16
2.0.1	Estrutura da Pasta do Projeto	16
2.0.2	Configuração dos Arquivos de Manifesto e Construção do Pacote	17
2.0.3	Entendendo a Anatomia do URDF: O que são Links e Joints?	20
3	Exemplo Prático de Código	25
3.0.1	Análise Linha por Linha do Código Exemplo	26
4	Automatização do Ambiente com Launch Files	28
4.0.1	O Papel dos Nós Fundamentais de Inicialização	28
4.0.2	Exemplo Prático: Script de Inicialização (<code>display.launch.py</code>)	28
4.0.3	Análise do Processo de Injeção Física	30
5	Adequação para o Isaac Sim	32
5.0.1	Como converter arquivo xacros em urdfs	32

1 PREPARAÇÃO DAS PARTES E MALHAS TRIDIMENSIONAIS

O *Unified Robot Description Format* (URDF) consiste em uma especificação em formato XML para a representação nativa de modelos robóticos no ecossistema ROS. O escopo deste manual abrange a padronização e boas práticas no desenvolvimento dessas estruturas.

Para a estruturação de um arquivo de montagem personalizado, recomenda-se a utilização do software de modelagem computacional *Blender*, embora certas ressalvas técnicas devam ser rigorosamente observadas para garantir um ótimo funcionamento de seu arquivo de simulação:

1. **Compatibilidade da Versão:** Orienta-se a utilização específica da versão 4.5.5 LTS, visto que variações no pipeline de desenvolvimento do software podem alterar a integridade da exportação de arquivos Collada (.dae) contendo propriedades de materiais e texturas de cor integradas;
2. **Consistência de Unidades de Medida:** Devido à natureza do *Blender* como um ecossistema voltado à modelagem 3D e Computação Gráfica (CGI), sua unidade métrica padrão de fábrica é o metro (com fator de escala igual a 1.0). Consequentemente, ao importar malhas geradas em milímetros a partir de softwares CAD de engenharia — como o *Autodesk Inventor* ou *Autodesk Fusion 360*, ocorre uma distorção dimensional direta. Por exemplo, um componente projetado com 23 mm de comprimento será interpretado erroneamente no ambiente do *Blender* como possuindo 23 m de extensão. Os procedimentos para a correção serão detalhados nas seções subsequentes;
3. **Alinhamento do Centro Geométrico (Origem):** Durante o processo de importação de arquivos externos, é comum que o ponto de origem ou centro geométrico do objeto (*origin point*) seja deslocado em relação aos eixos globais. Cabe ao desenvolvedor reposicionar e centralizar essa origem manualmente no software, uma vez que a localização correta deste referencial é um requisito mandatório para determinar os eixos de rotação das articulações (*joints*) e a distribuição correta das propriedades inerciais no arquivo URDF final;
4. **Propósito do Ambiente de Modelagem:** Ressalta-se que o papel do *Blender* neste pipeline restringe-se à organização espacial dos componentes, verificação de escala e ajustes de centros geométricos. Para a modelagem e concepção geométrica de peças mecânicas complexas, preconiza-se a utilização dedicada de softwares CAD industriais de livre escolha do projetista;
5. **Formatos de Exportação Intermediários:** Ao exportar os componentes a partir do software CAD de engenharia para o *Blender*, sugerem-se duas abordagens distintas dependendo do fluxo de trabalho: a exportação em formato .stl (caso a personalização de cores e materiais seja realizada posteriormente dentro do próprio *Blender*) ou em formato .obj

(quando for desejável preservar e importar diretamente as definições de cores e aparências estruturadas na modelagem original).

1.1 Organização Estrutural e Preparação de Malhas no Blender

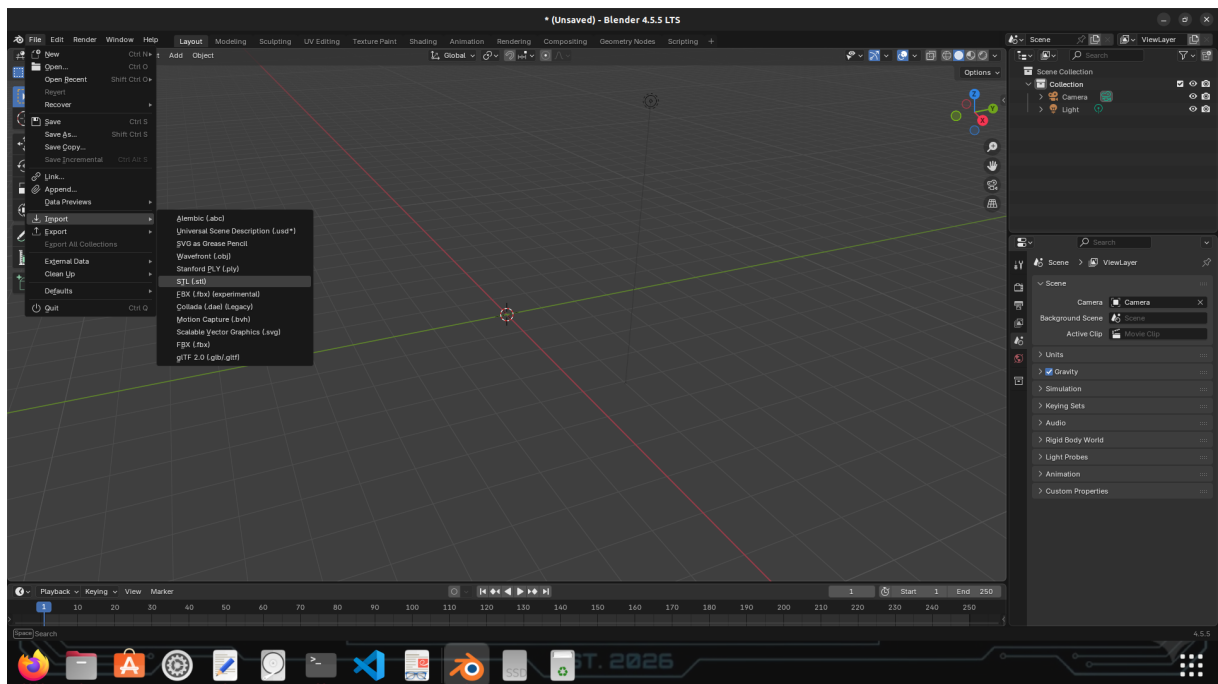
Para que um modelo 3D seja importado corretamente no arquivo URDF (*Unified Robot Description Format*), não basta apenas desenhar ou importar as peças. O ROS 2 e os simuladores (como o Gazebo) exigem rigor técnico quanto à **escala**, à **orientação** e, principalmente, à **posição da origem (ponto pivô)** de cada *link*.

Abaixo, descrevemos o processo ideal de importação, correção de escala, alinhamento de centros geométricos (origens) e exportação utilizando o Blender.

1.1.1 Passo 1: Importação e Configuração do Ambiente

O primeiro passo consiste em trazer as partes individuais de cada elemento do robô para a cena do Blender (conforme demonstrado na **Figura 1.1**).

Figura 1.1: Interface de importação no Blender



Ao importar cada peça, duas ações iniciais são fundamentais para o fluxo de trabalho:

1. **Visualização:** Mude o modo de exibição para **Viewport Shading** (conforme demonstrado na **Figura 1.2**) para visualizar melhor as superfícies e texturas do modelo.
2. **Painel de Propriedades:** Pressione a tecla **N** no teclado para habilitar a *Sidebar* (painel lateral). Na aba *Transform*, você terá acesso imediato e em tempo real às informações de

Localização (Location), Rotação (Rotation), Escala (Scale) e Dimensões (Dimensions) do objeto selecionado (conforme demonstrado na **Figura 1.3**).

Figura 1.2: Mudança de exibição

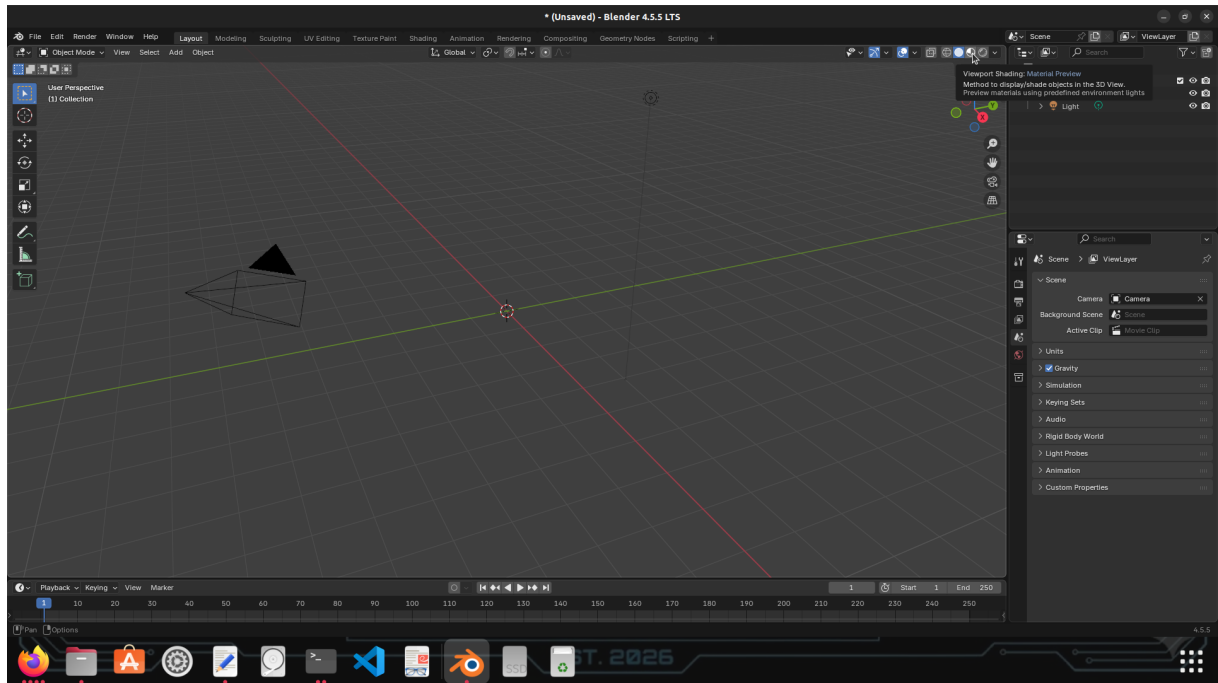
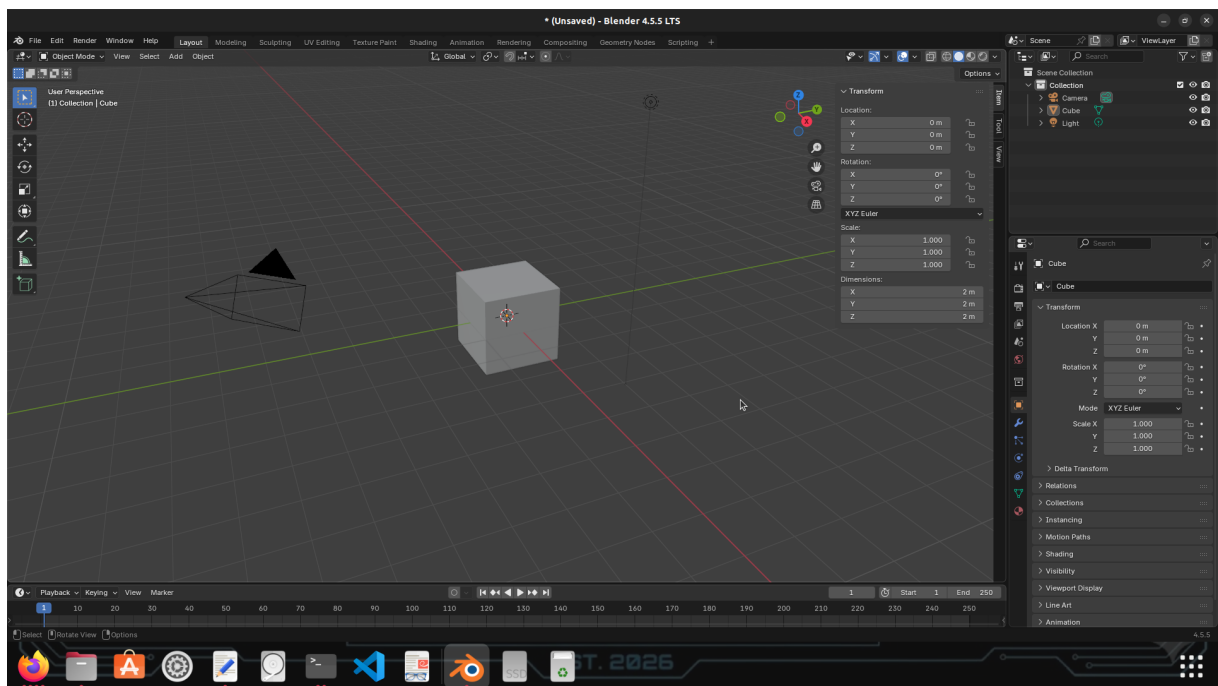


Figura 1.3: Painel lateral habilitado

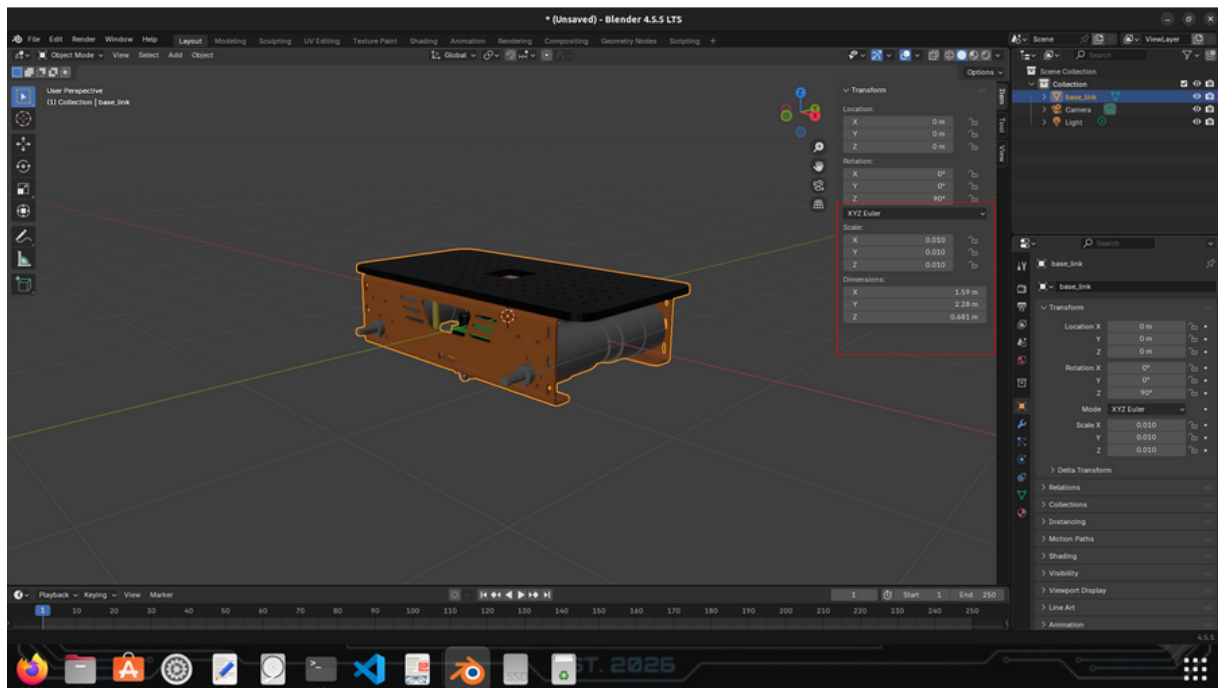


1.1.2 Passo 2: O Desafio da Escala

O Blender e os softwares de CAD frequentemente apresentam incompatibilidades de escala na importação (geralmente devido à conversão de milímetros para metros). Para resolver isso, verifique o campo *Dimensions*. Se a peça estiver fora do tamanho real, adote uma das seguintes estratégias de recomendação:

- Mude o valor do campo **Scale** diretamente para 0.001 no Blender.
- **Alternativa:** Aplique a escala de 0.01 no Blender e, posteriormente, no arquivo URDF, defina a tag de escala do *visual/collision* como 0.1 (conforme demonstrado na **Figura 1.4**).

Figura 1.4: Peça com escala adequada



1.1.3 Passo 3: Correção do Centro Geométrico e Pontos de Articulação (Origem)

Por padrão, a importação pode deslocar o ponto de origem do objeto (o ponto laranja destacado na **Figura 1.5**). No URDF, esse ponto laranja será a **origem do link** e a posição exata onde a junta (*joint*) se conectará. Por isso, ele deve ser posicionado com precisão.

Figura 1.5: Ponto de origem do objeto



Para Partes Fixas

O ponto laranja deve ficar localizado exatamente no centro geométrico da peça.

Para Partes Móveis (Links de Braços Robóticos, Rodas, etc.)

O ponto laranja **deve** ser posicionado exatamente no eixo de rotação ou translação da peça. Por exemplo, como demonstrado na **Figura 1.6**, se a junta realiza rotações em relação ao eixo Y (linha verde), o ponto laranja deve ser posicionado no início da conexão do cotovelo do link. O mesmo vale para rodas (movimento angular), onde o ponto deve ficar no centro exato do eixo da roda (**Figura 1.7**).

Figura 1.6: Junta de braço robótico eixo de rotação em Y

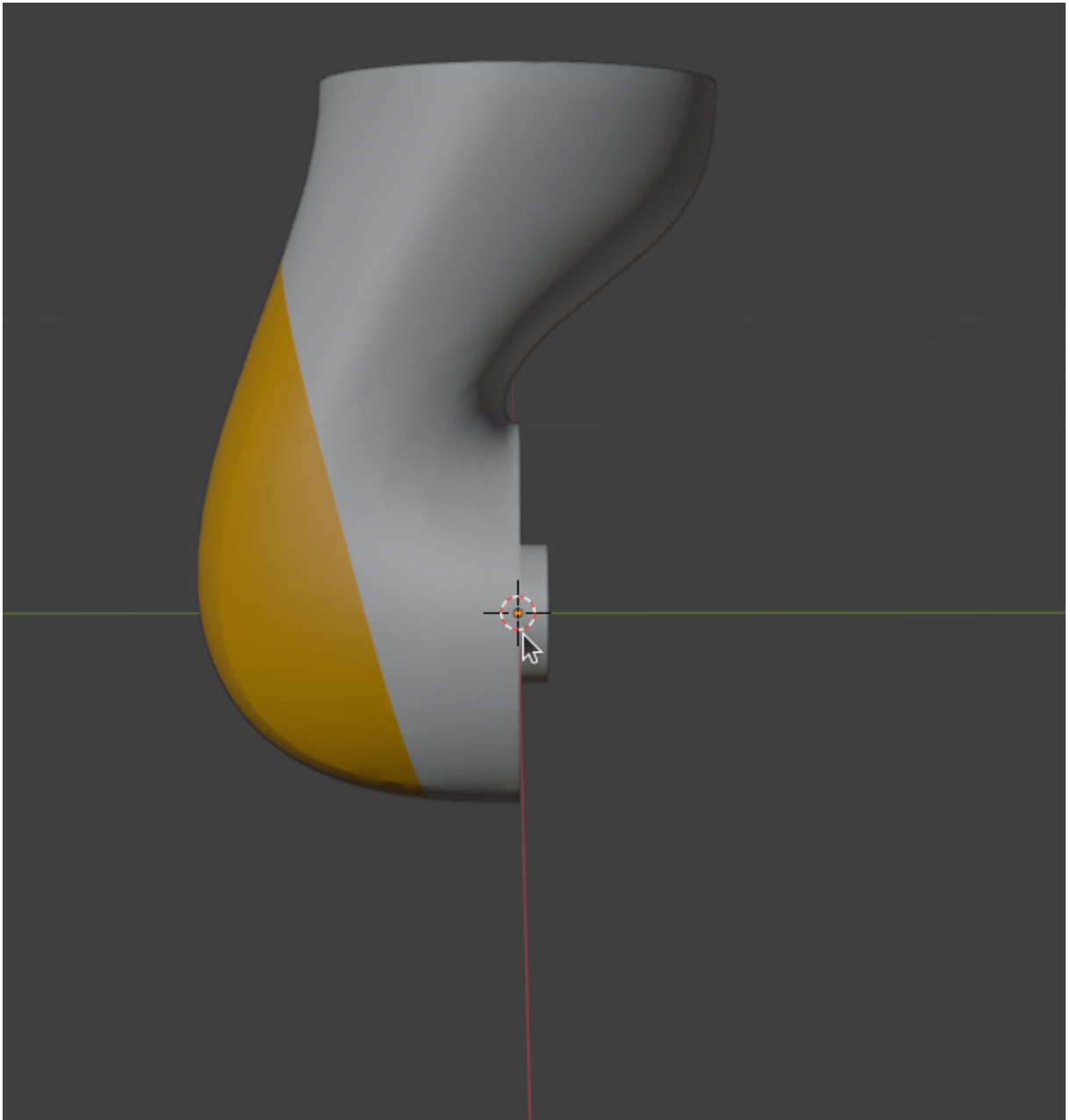
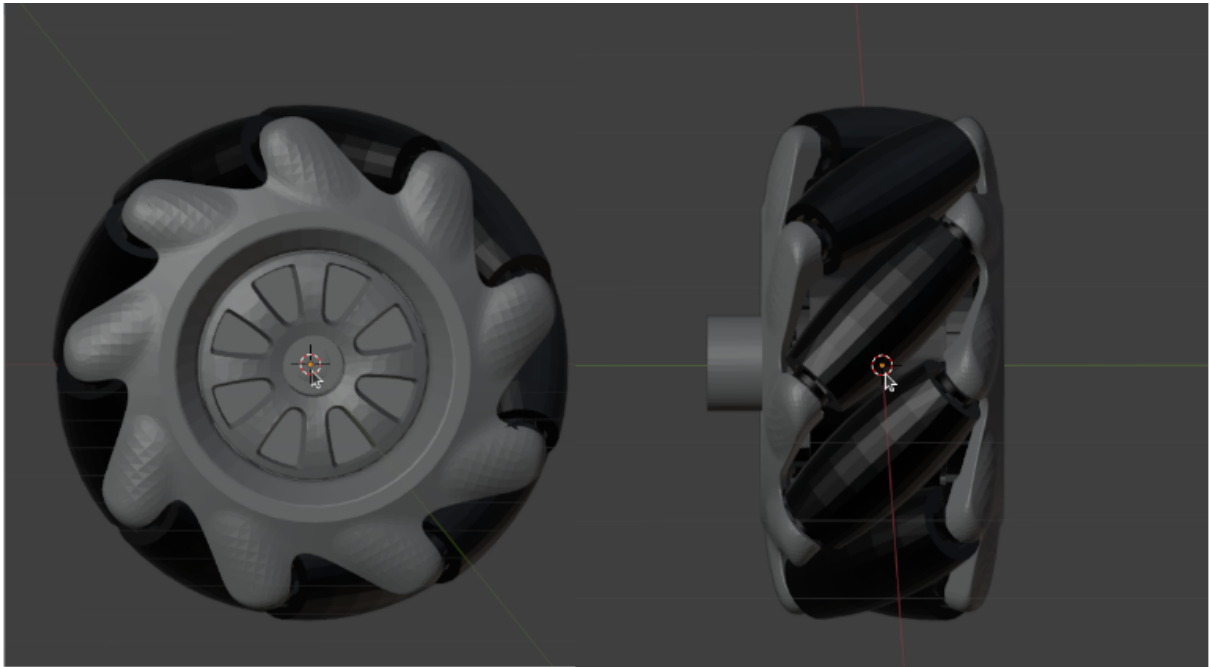


Figura 1.7: Ponto de origem da roda



Procedimento para movimentar a malha mantendo a origem fixa

Para mover a geometria sem alterar a posição global do ponto laranja, siga os passos abaixo:

1. **Entrar no Modo de Edição:** Selecione a peça específica e pressione **TAB** para alternar do *Object Mode* para o *Edit Mode*.
2. **Selecionar Toda a Malha:** Pressione **A** para selecionar todos os vértices, arestas e faces. A peça ficará inteiramente iluminada.
3. **Mover a Malha em Relação à Origem:** Clique na tecla **G** seguido do eixo desejado (**X**, **Y** ou **Z**) para movimentar a geometria da peça. Mova-a até que o ponto laranja fixo fique exatamente na posição de movimento ou centro desejado. Pressione **TAB** para sair do Modo de Edição ao finalizar.

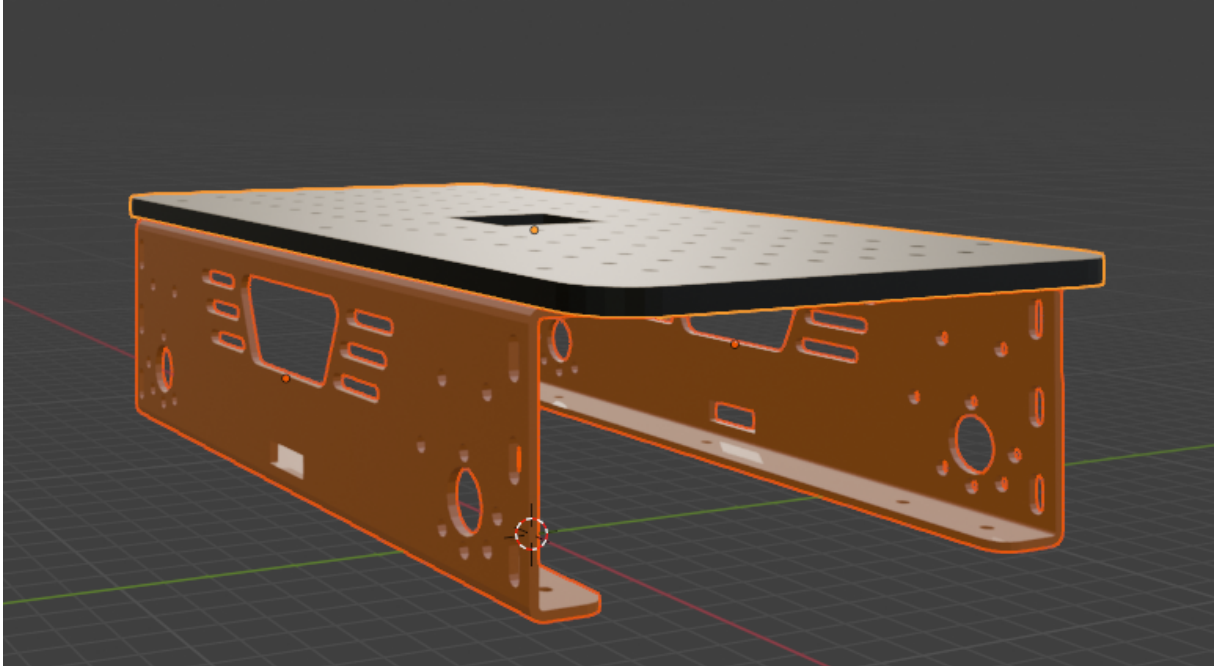
1.1.4 Passo 4: Junção de Peças Estáticas (Join)

Após configurar as peças individuais, podemos agrupar os elementos que se mantêm fixos entre si (ou seja, que não possuem movimento relativo). Na **Figura 1.8**, por exemplo, temos duas laterais laranjas e uma placa retangular que sempre acompanharão o movimento do robô unidas; elas não possuem juntas entre si, logo, podem se tornar um único objeto.

Atenção Importante: Ao juntar peças, o ponto de origem (ponto laranja) do novo objeto unificado ficará localizado na posição do **último objeto individual que você selecionou** (o objeto ativo, que fica com o contorno amarelo claro, conforme

demonstrado na **Figura 1.8**). Certifique-se de que o último objeto selecionado seja aquele que detém a origem correta para o URDF, ou readeque a origem após a junção.

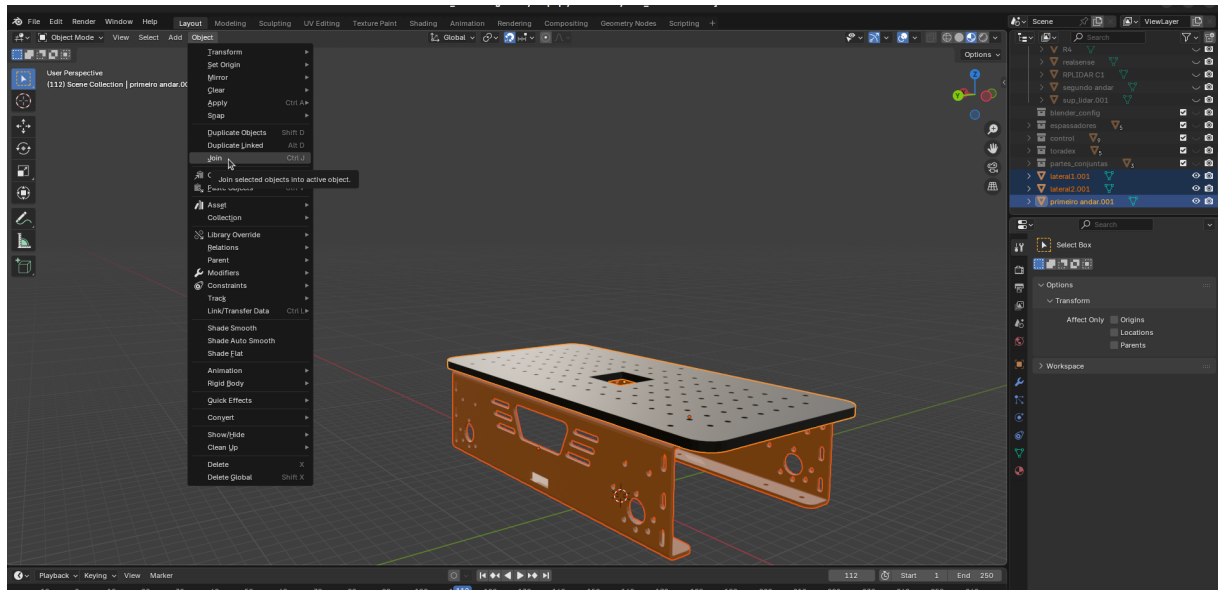
Figura 1.8: Seleção das peças (ultima selecionada placa retangular)



Para realizar a junção de forma segura (preservando os originais se necessário):

1. Selecione todas as partes que serão unidas.
2. Crie uma cópia de segurança pressionando **Ctrl + C** e **Ctrl + V**.
3. Com as cópias selecionadas, vá até o menu superior **Object** → **Join** (ou utilize o atalho **Ctrl + J**), conforme demonstrado na **Figura 1.9**.

Figura 1.9: Peças unificadas



1.1.5 Passo 5: Exportação para COLLADA (.dae)

Com as peças individuais e os conjuntos estáticos devidamente ajustados, o passo final é a exportação para o formato .dae, amplamente utilizado no URDF por suportar materiais e malhas complexas de forma leve.

Para garantir que o URDF posicione a peça corretamente, exportamos a malha alinhada à origem global (0, 0, 0) do Blender:

1. Selecione a peça desejada e mude temporariamente sua localização (*Location*) na *Sidebar* para 0, 0, 0.
2. Vá em **File** → **Export** → **Collada (.dae)**, demonstrado na **Figura 1.10**.
3. Escolha a pasta de destino do seu pacote ROS .
4. No painel de exportação, habilite a opção **Selection Only** (Apenas Seleccionados) para exportar estritamente a peça ativa, demonstrado na **Figura 1.11**.
5. Após salvar, retorne a peça para sua posição original na cena do Blender utilizando o atalho **Ctrl + Z**.
6. Repita o mesmo processo para todas as demais peças do robô.

Figura 1.10: Exportando o arquivo .dae

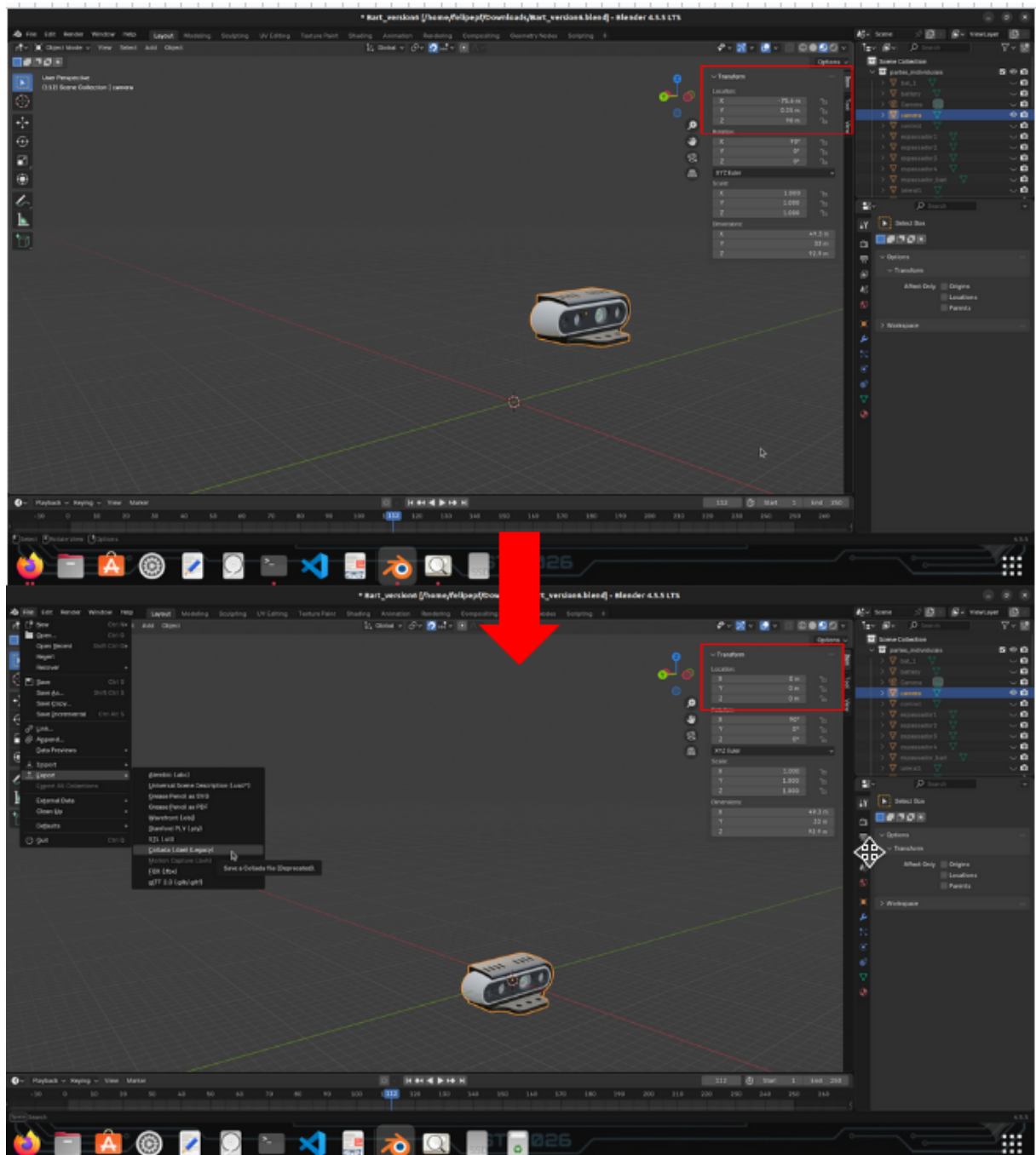
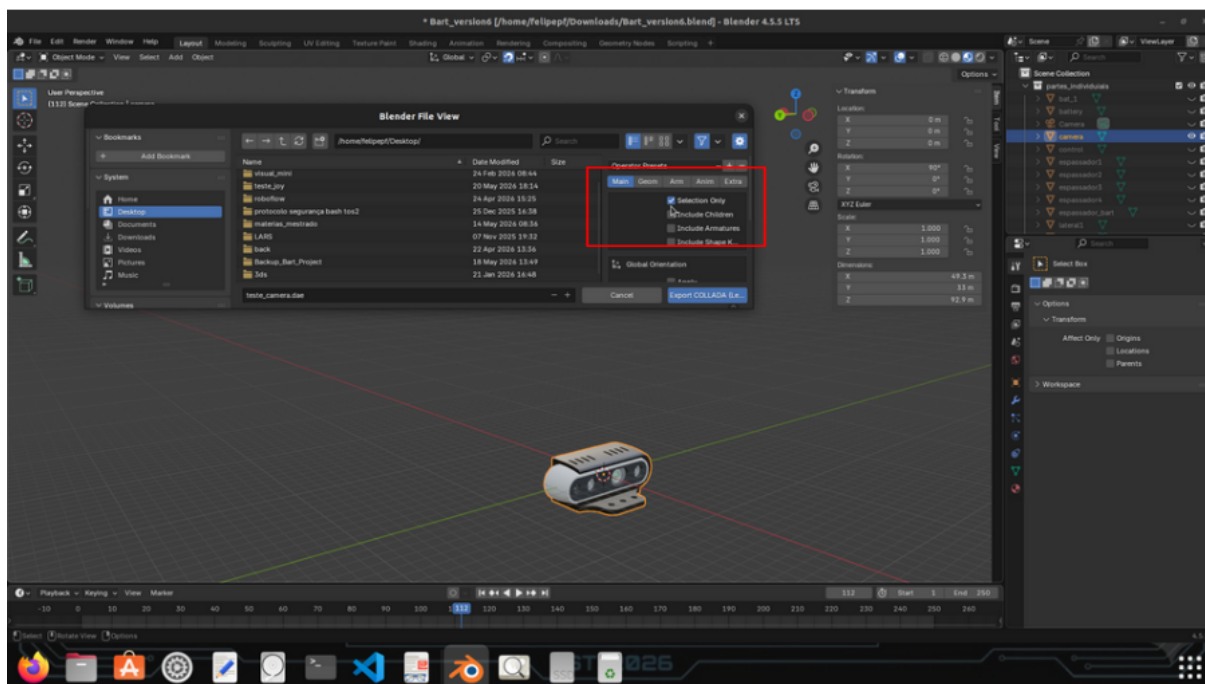


Figura 1.11: Tela de exportação



Com todas as peças adequadas, com escalas corrigidas, origens bem posicionadas e exportadas estruturalmente, seu modelo está pronto para ser mapeado no arquivo URDF.

1.1.6 Importação, Otimização e Customização de Arquivos STL

Muitas vezes, as malhas estruturais do robô são projetadas em softwares de CAD e exportadas no formato STL (*Stereolithography*). Embora seja um formato popular para manufatura e impressão 3D, o STL apresenta algumas limitações críticas para simulações no ROS (como falta de suporte a cores/materiais nativos e malhas excessivamente densas).

Nesta seção, abordaremos o procedimento correto para importar, otimizar a topologia e customizar visualmente seus arquivos STL dentro do Blender.

Passo 1: Importação do Arquivo STL

Para trazer o seu modelo CAD para o ambiente de preparação:

1. Vá no menu superior e clique em **File** → **Import** → **Stl (.stl)** (conforme demonstrado na **Figura 1.1**).
2. Navegue até a pasta onde o arquivo está salvo, selecione-o e clique em *Import STL*.

Nota de Escala: Assim que o arquivo for importado, lembre-se de aplicar os conceitos de validação de dimensões e ajuste de escala abordados na seção anterior, garantindo que o objeto não fique gigante ou invisível na cena.

Passo 2: Customização Estética (Materiais e Cores)

Como arquivos STL não carregam informações de cor, o modelo importado parecerá inteiramente cinza. Para dar uma identidade visual profissional ao seu robô:

1. Selecione a peça desejada e vá até a aba **Material Properties** (ícone de uma esfera em tom de xadrez no menu lateral direito).
2. Clique em **New** para criar um novo material, conforme demonstrado na **Figura 1.12**.
3. No campo **Base Color**, selecione a cor desejada para o seu *link*, conforme demonstrado na **Figura 1.13**.
4. Caso queira atribuir cores diferentes a partes específicas do objeto, basta entrar no modo de edição (*Edit Mode*), selecionar a face ou as faces desejadas e acessar a aba **Material Properties**. Em seguida, clique em **New** (ou no ícone de +), escolha a cor desejada no campo **Base Color** e, por fim, clique em **Assign**, conforme demonstrado na **Figura 1.14**.

Figura 1.12: Criação de um novo material

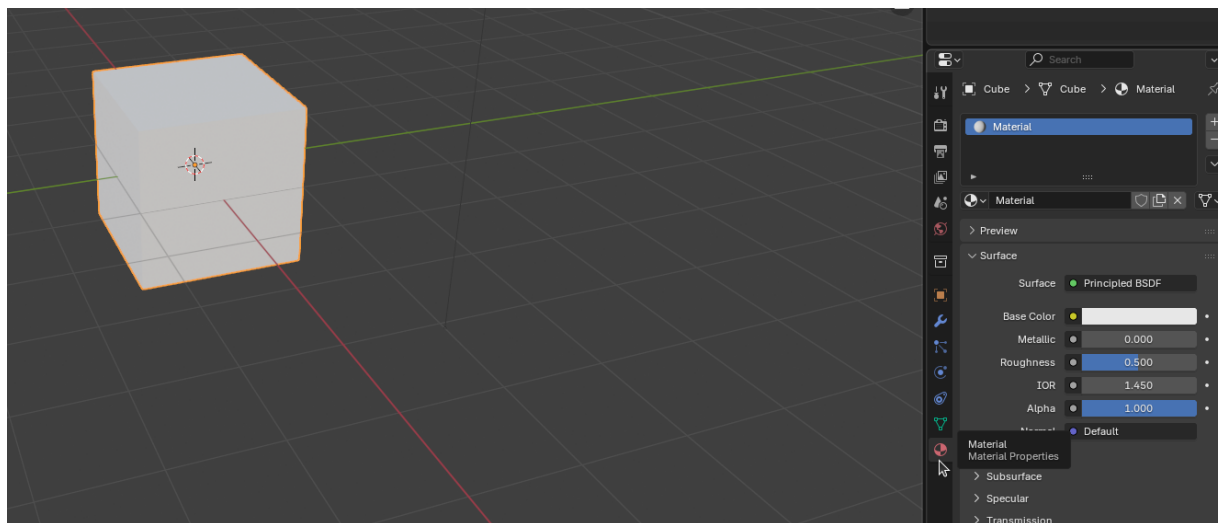


Figura 1.13: Modificação da cor com um novo material

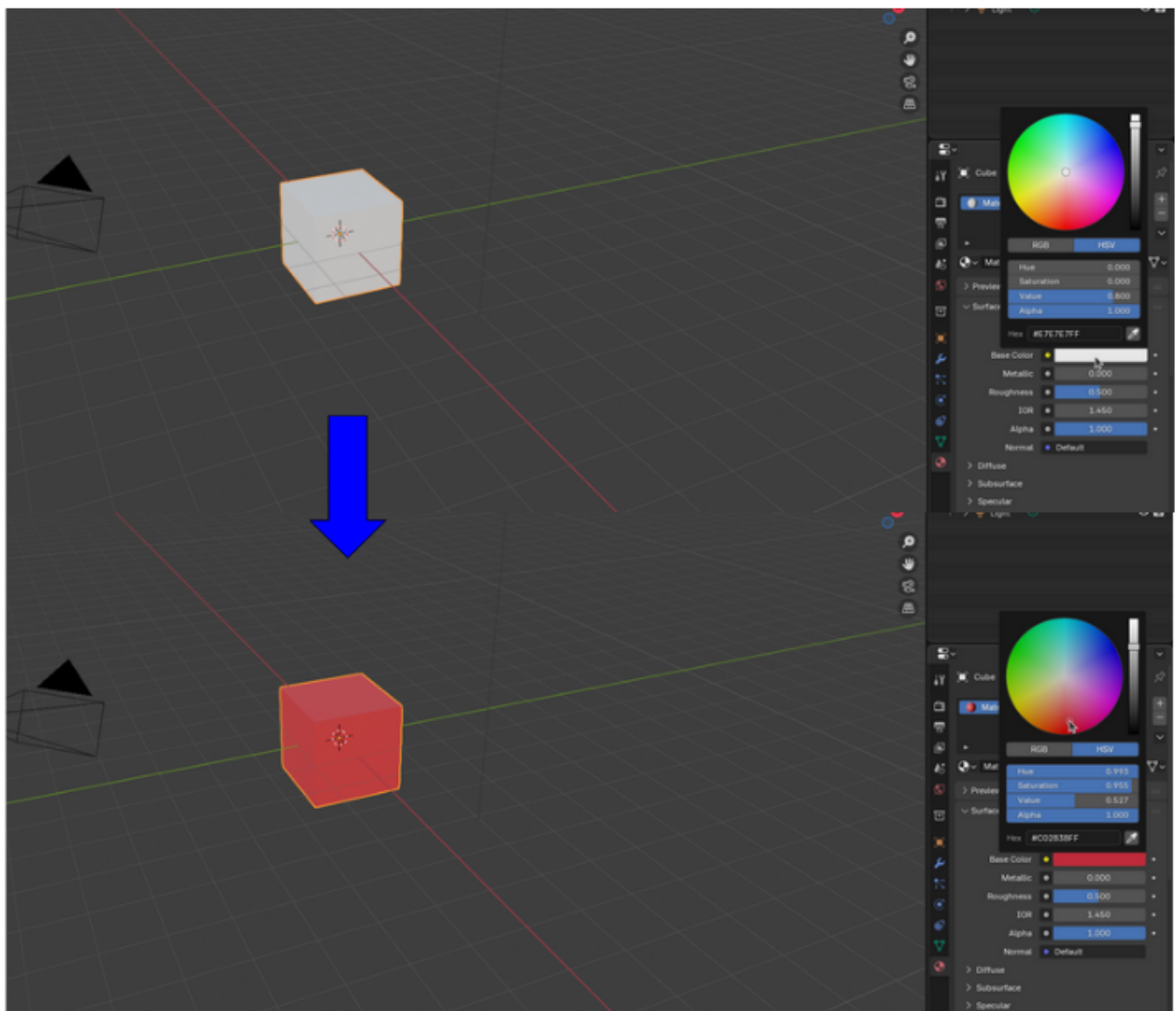
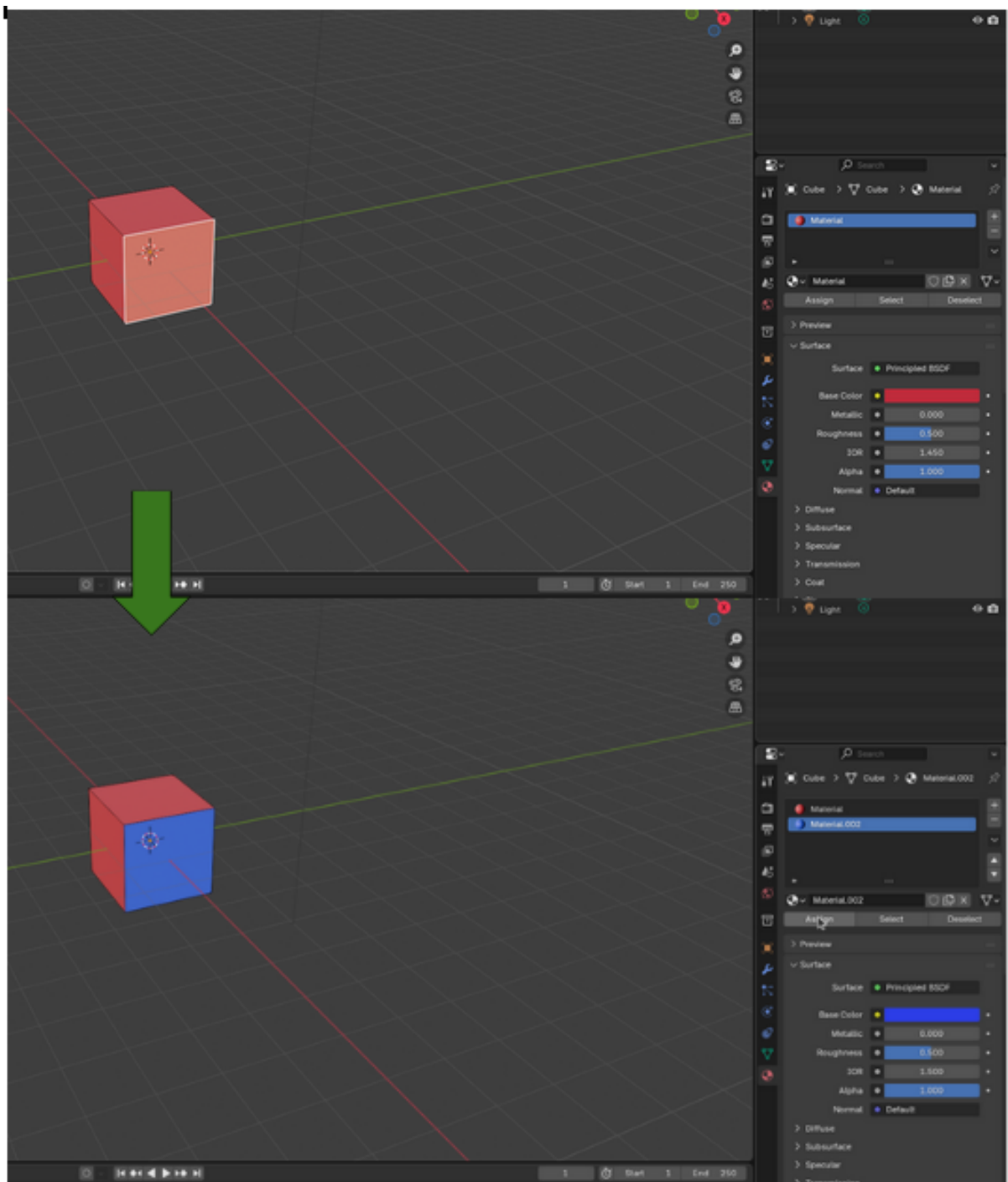


Figura 1.14: Implementação de cores diferentes



2 CRIAÇÃO DA PASTA, MONTAGEM DO URDF E CONFIGURAÇÃO DO AMBIENTE

Depois de preparar, otimizar e exportar todas as malhas geométricas no formato .dae dentro do Blender, o próximo passo essencial é integrá-las à estrutura de desenvolvimento do ROS 2.

Neste capítulo, aprenderemos como criar a árvore de diretórios correta para o robô, como estruturar o esqueleto inicial do arquivo URDF e como configurar o ambiente de desenvolvimento para que o sistema reconheça o seu modelo.

2.0.1 Estrutura da Pasta do Projeto

No ROS 2, os modelos de robôs devem ser organizados dentro de um pacote dedicado, por exemplo *nome_do_robo_project*. Para isso, você deve navegar até o diretório de código-fonte (*source*) do seu *workspace* e utilizar a ferramenta de linha de comando do ROS 2 para gerar o pacote estruturado com suporte a scripts Python e licença Apache-2.0.

O procedimento passo a passo no terminal consiste nos seguintes comandos:

1. Navegar até o workspace:

```
cd ~/workspace_ws/src
```

2. **Criar o pacote ROS 2:** Execute o comando abaixo substituindo *nome_do_robo_project* pelo nome do seu projeto. O parâmetro `-build-type ament_python` garante a compatibilidade com scripts de inicialização (*launch files*) e automações em Python, enquanto o `-license` define formalmente os direitos de uso:

```
ros2 pkg create nome_do_robo_project \
  --build-type ament_python \
  --license Apache-2.0
```

3. **Criar a árvore de diretórios necessária:** Entre na pasta do pacote recém-criado e utilize o comando `mkdir` para gerar todas as subpastas estruturais demandadas para a organização do robô, incluindo os subdiretórios de malhas visuais:

```
cd nome_do_robo_project
mkdir -p config launch rviz meshes/visual world urdf script
```

Com isso, a estrutura base do seu pacote estará pronta e padronizada. A pasta `meshes/visual` deverá receber os arquivos `.dae` exportados do Blender, a pasta `urdf` abrigará a descrição estrutural do robô, e os diretórios `launch` e `rviz` conterão os scripts de inicialização e as telas de configuração visual do ecossistema, respectivamente.

2.0.2 Configuração dos Arquivos de Manifesto e Construção do Pacote

Após criar a estrutura física de diretórios, é fundamental configurar os arquivos de build do ROS 2. Em pacotes que utilizam o sistema de construção *ament_python*, a gerência de dependências de software e o mapeamento de arquivos para o diretório de instalação do sistema dependem diretamente da edição correta dos arquivos *package.xml* e *setup.py*, além da inclusão do arquivo de manifesto para o simulador Gazebo, o *model.config*.

Passo 1: Gerenciamento de Dependências no *package.xml*

O arquivo *package.xml* define formalmente as propriedades do pacote e lista todas as dependências de execução (*exec_depend*) necessárias para que nós de teleoperação, publicadores de estados e sistemas de controle funcionem de forma integrada. Abra o arquivo localizado na raiz do seu pacote e configure-o adicionando as dependências de robótica e simulação, conforme demonstrado no exemplo a seguir:

```
<?xml version="1.0"?>
<package format="3">
  <name>nome do seu pkg</name>
  <version>0.0.0</version>
  <description>rapida descricao</description>

  <maintainer email="seu email">seu nome</maintainer>
  <license>Apache-2.0</license>

  <!-- Build system -->
  <buildtool_depend>ament_python</buildtool_depend>

  <!-- Runtime dependencies -->
  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>nav_msgs</exec_depend>
  <exec_depend>tf2</exec_depend>
  <exec_depend>tf2_ros</exec_depend>

  <!-- Robot / visualization -->
```

```

<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>joint_state_publisher</exec_depend>
<exec_depend>xacro</exec_depend>
<exec_depend>rviz2</exec_depend>
<exec_depend>ros2_control</exec_depend>
<exec_depend>ros2_controllers</exec_depend>
<exec_depend>controller_manager</exec_depend>
<exec_depend>diff_drive_controller</exec_depend>
<exec_depend>joint_state_broadcaster</exec_depend>
<exec_depend>teleop_twist_keyboard</exec_depend>
<exec_depend>gazebo_ros2_control</exec_depend>

<!-- Testing -->
<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
<test_depend>python3-pytest</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

Passo 2: Mapeamento de Recursos no *setup.py*

Diferente do ecossistema CMake, um pacote Python necessita que o script *setup.py* aponte explicitamente onde estão localizados os arquivos de malhas, configurações do RViz, mapas e mundos do simulador. Isso garante que a ferramenta *colcon build* consiga exportar e indexar esses recursos no diretório compartilhado do ROS 2 (*share/*).

Abra o arquivo *setup.py* e reestruture o campo *data_files*, utilizando a biblioteca *glob* para coletar dinamicamente o conteúdo de cada subpasta do projeto:

```

from setuptools import setup, find_packages
import os
from glob import glob

package_name = 'nome do seu pkg'

setup(
    name=package_name,

```

```

version='0.0.0',

# Pacote descritivo -> nao precisa de modulos Python para a instala
packages=find_packages(),

data_files=[
    # Registro do pacote no ament index
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),

    ('share/' + package_name, ['model.config']),

    # package.xml
    ('share/' + package_name,
     ['package.xml']),

    # Launch files
    (os.path.join('share', package_name, 'launch'),
     glob('launch/*.py')),

    # URDF / XACRO
    (os.path.join('share', package_name, 'urdf'),
     glob('urdf/*')),

    (os.path.join('share', package_name, 'config'),
     glob('config/*')),

    # Meshes VISUAL
    (os.path.join('share', package_name, 'meshes', 'visual'),
     glob('meshes/visual/*.*')),

    # Worlds
    (os.path.join('share', package_name, 'worlds'),
     glob('worlds/*.world')),

    # Rviz
    (os.path.join('share', package_name, 'rviz'),

```

```

        glob('rviz/*.rviz')),
    ],

    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='seu nome',
    maintainer_email='seu email',
    description='uma rapida descrição',
    license='Apache License 2.0',
    entry_points={
        'console_scripts': [],
    },
)

```

Passo 3: Integração com o Simulador através do *model.config*

Para que simuladores como o Gazebo Classic identifiquem o modelo do robô de forma automática dentro do seu banco de modelos locais (*GAZEBO_MODEL_PATH*), o diretório do pacote precisa conter um arquivo de manifesto simplificado chamado exatamente *model.config*.

Crie este arquivo na raiz do seu pacote para declarar o ponto de entrada do arquivo URDF processado e os dados:

```

<?xml version="1.0"?>
<model>
  <name>nome do robo</name>
  <version>1.0</version>
  <sdf version="1.7">urdf/processed/bart_version1.urdf</sdf>
  <author>
    <name>seu nome</name>
    <email>seu email</email>
  </author>
  <description>
    uma rapida descrição
  </description>
</model>

```

2.0.3 Entendendo a Anatomia do URDF: O que são Links e Joints?

Para quem está iniciando na robótica, um arquivo URDF pode parecer um emaranhado complexo de códigos XML. No entanto, sua lógica é puramente mecânica. O URDF descreve um robô como uma árvore cinemática composta por duas entidades fundamentais:

- **Links (Elos/Corpos):** Representam as partes rígidas e físicas do robô (ex: o chassi, um braço de alumínio, uma roda).
- **Joints (Juntas/Articulações):** Representam os pontos de conexão que unem um Link ao outro, ditando como eles podem se mover entre si (ex: um rolamento, uma dobradiça, um motor).

Cada **Link** individual é internamente dividido em três blocos essenciais que dizem ao computador como a peça se parece, onde ela bate e como ela pesa: **Visual**, **Collision** e **Inertial**. A seguir, detalhamos a estrutura de cada um deles.

1. O Bloco Visual (<visual>) — A Aparência do Robô

O bloco <visual> serve exclusivamente para os nossos olhos e para as ferramentas de renderização gráfica (como RViz2 e as câmeras do simulador). Ele define **o que nós vemos** na tela.

- **O que colocar aqui:** É onde inserimos as malhas complexas, bonitas e texturizadas que exportamos do Blender (.dae).
- **Atenção técnica:** Este bloco não possui propriedades físicas. Se você colocar uma malha visual gigante e pesada, o robô parecerá robusto na tela, mas para a física do simulador, ele ainda será invisível e intangível.

2. O Bloco de Colisão (<collision>) — O Corpo Físico

O bloco <collision> define a geometria (ODE, Bullet, PhysX) que será utilizada para calcular os impactos, contatos com o solo e barreiras do ambiente. Ele determina **onde o robô bate**.

- **O que colocar aqui:** Em vez de usar a malha complexa do Blender (que exigiria bilhões de cálculos matemáticos para computar uma colisão), usamos **geometrias primitivas simples** (como caixas <box>, cilindros <cylinder> ou esferas <sphere>) que envolvem a peça aproximada.
- **Por que fazer isso:** Se o chassi do robô colidir com uma parede, o simulador irá calcular o impacto baseado na caixa invisível do bloco <collision>, poupando processamento computacional e evitando que o robô atravessasse objetos ou caia através do chão da simulação.

3. O Bloco Inercial (<inertial>) — A Física e a Massa

Este é o bloco que costuma gerar mais dúvidas em quem está começando. O bloco <inertial> dita as forças da gravidade e da aceleração sobre o robô. Ele determina **o quanto difícil é mover ou parar a peça**. Ele é composto por três parâmetros matemáticos:

- **Massa (<mass>):** O peso real do componente em quilogramas (kg). Se você disser que o chassi pesa 10 kg, as forças aplicadas pelos motores precisarão ser proporcionalmente maiores para tirá-lo da inércia.
- **Origem (<origin>):** Indica a coordenada X, Y, Z do **Centro de Massa** (ou centro de gravidade) da peça em relação à origem do Link. Se o centro de massa estiver deslocado para o lado esquerdo, o robô tenderá a tombar para a esquerda sob efeito da gravidade.
- **Matriz de Inércia (<inertia>):** Representada pelos momentos de inércia rotacionais (I_{xx}, I_{yy}, I_{zz}). Enquanto a massa mede a resistência ao movimento em linha reta, a inércia mede a **resistência do objeto a girar** em torno de seus próprios eixos.

4. A Estrutura da Junta (<joint>) — O Vínculo de Movimento

Se os Links são os ossos, a <joint> é a articulação. Uma junta cria uma relação de dependência hierárquica entre duas peças: o Link Pai (**parent**) e o Link Filho (**child**).

A estrutura básica e o significado de suas tags são explicados abaixo de forma genérica:

- **parent** (Link Pai): É a peça que serve de referência estática ou base. Se o Link Pai se mover, o Link Filho se moverá rigidamente junto com ele.
- **child** (Link Filho): É a peça móvel (ou acoplada) que recebe o movimento gerado pela junta.
- **origin** (Origem da Junta): Define a posição exata (coordenadas X, Y, Z em metros) e a rotação (ângulos *Roll*, *Pitch*, *Yaw* em radianos) onde o ponto de articulação (aquele ponto laranja que configuramos no Blender) ficará posicionado em relação ao centro do Link Pai.
- **axis** (Eixo de Movimento): Especifica o vetor unitário (X, Y, Z) ao redor do qual (ou ao longo do qual) o movimento acontece. Por exemplo, se o eixo for $x_{yz}="0 \ 1 \ 0"$, significa que a roda ou articulação girará estritamente em torno do eixo Y local da junta.
- **limit** (Limites Físicos): Restringe a operação da junta definindo o torque máximo suportado pelo motor (*effort*) e a velocidade máxima angular ou linear permitida (*velocity*).

O URDF possui 6 tipos primitivos de juntas em tópicos didáticos:

- **fixed** (Fixa):
 - **O que indica:** Bloqueia totalmente qualquer movimento relativo entre as duas peças. Elas se comportam como se estivessem soldadas ou parafusadas rigidamente.
 - **Uso comum:** Conectar sensores (como câmeras ou sensores LiDAR) ao chassi principal, ou unir subestruturas estáticas da carcaça do robô.

- **continuous** (Contínua):
 - **O que indica:** Permite um movimento de rotação pura em torno de um único eixo especificado, de forma ****ilimitada**** (pode girar 360 graus infinitamente). Não possui travas de fim de curso.
 - **Uso comum:** Modelagem de rodas de robôs móveis (rodas diferenciais, omnidirecionais, etc.) ou polias de transmissão.
- **revolute** (Rotacional / Servo):
 - **O que indica:** Permite um movimento de rotação em torno de um único eixo, mas possui ****limites angulares estritos**** de início e fim de curso (ângulos mínimo e máximo expressos em radianos no código).
 - **Uso comum:** Articulações de braços robóticos industriais (ombro, cotovelo, pulso), garras de manipulação e sistemas de movimentação de câmera (*pan-tilt*).
- **prismatic** (Prismática / Linear):
 - **O que indica:** Em vez de girar, permite um movimento de ****translação linear**** (deslocamento em linha reta) ao longo de um eixo específico. Também exige a configuração de limites físicos de curso (mínimo e máximo expressos em metros).
 - **Uso comum:** Atuadores lineares, pistões pneumáticos, elevadores de carga de robôs logísticos e eixos de impressoras 3D.
- **planar** (Planar):
 - **O que indica:** Permite que a peça filho deslize e se movimente livremente sobre a superfície de um plano bidimensional. Ela ganha três graus de liberdade dentro daquele plano: translação em *X*, translação em *Y* e rotação (guinada/*yaw*) em torno do eixo *Z*.
 - **Uso comum:** Representar o deslocamento simplificado de bases móveis omnidirecionais diretamente sobre o solo sem precisar calcular o atrito roda a roda.
- **floating** (Flutuante):
 - **O que indica:** Permite movimento completamente livre no espaço tridimensional em todas as direções possíveis, ou seja, libera todos os 6 graus de liberdade (3 translações espaciais e 3 rotações angulares). Não há nenhuma restrição geométrica entre o pai e o filho.
 - **Uso comum:** Utilizado para modelar robôs aéreos (como drones e quadricópteros) ou a base flutuante de robôs humanoides e quadrúpedes antes de tocarem os pés no chão.

Com a combinação correta desses 6 tipos de juntas e a adição dos plugins correspondentes para ler sensores e controlar os motores, você consegue simular e programar desde um simples carrinho de controle remoto até robôs industriais e humanoides de altíssima complexidade no ROS 2!

1. O que são Plugins de Simulação? — A Ponte de Software

Imagine que você projetou o corpo do robô perfeitamente no URDF. Ele tem peso, tem rodas e tem uma câmera instalada. Porém, se você simplesmente jogá-lo dentro do simulador (como o Gazebo), ele se comportará como uma estátua: as rodas não girarão quando você enviar um comando de velocidade, e a câmera não gerará nenhuma imagem para o seu código de Visão Computacional.

É aqui que entram os **Plugins**. Um plugin é um pedaço de software (geralmente uma biblioteca compilada em C++ com extensão `.so`) que atua como uma **ponte eletrônica virtual**. Ele conecta o motor de física do simulador aos nós e tópicos do ROS 2.

Os plugins são divididos essencialmente em duas categorias funcionais:

1. **Plugins de Atuação e Movimento (Drivers Virtuais):** Eles simulam a eletrônica de controle e os motores do robô. Na prática, esse plugin fica “escutando” os tópicos de comando de velocidade do ROS 2 (como o `/cmd_vel`). Quando o usuário envia uma ordem de movimentação pelo teclado ou por um algoritmo de navegação, o plugin traduz esse comando em forças físicas e velocidades aplicadas diretamente nas juntas das rodas dentro da simulação. Além disso, ele calcula a **Odometria** (a estimativa de onde o robô está no espaço) e publica essa informação de volta para o ROS 2.
2. **Plugins de Sensores (Hardware Virtual):** Eles simulam o comportamento dos sensores físicos reais capturando os dados gerados dentro do mundo virtual.
 - *Exemplo de Câmeras RGB-D:* O plugin lê a matriz de pixels do ambiente virtual do simulador, calcula a profundidade tridimensional de cada objeto e empacota esses dados em formato de imagens brutas e *Point Clouds* (nuvens de pontos 3D). Ele então publica esses dados nos tópicos padrão do ROS 2, permitindo que algoritmos de processamento de imagem ou SLAM operem exatamente da mesma forma que operariam com o hardware real conectado ao computador.

Com essa fundação conceitual clara, ler qualquer código URDF se torna intuitivo: você sempre estará mapeando o que o robô exibe visualmente, o formato simplificado do seu corpo físico, o seu peso/distribuição de massa, e as regras mecânicas de como suas partes se articulam.

3 EXEMPLO PRÁTICO DE CÓDIGO

Abaixo apresenta-se a declaração mínima essencial para inicializar um robô no interpretador. Este exemplo prático demonstra como conectar um link de chassi a uma roda utilizando uma junta contínua, servindo como modelo base para compreender a sintaxe do URDF na prática:

```
<?xml version="1.0"?>
<robot name="robo_minimalista" xmlns:xacro="http://ros.org/wiki/xacro">

  <!-- 1. LINK VIRTUAL DE PROJEÇÃO NO SOLO -->
  <link name="base_footprint"/>

  <!-- 2. JUNTA FIXA DE CONEXÃO DA BASE -->
  <joint name="base_joint" type="fixed">
    <parent link="base_footprint"/>
    <child link="chassi_link"/>
    <origin xyz="0.0 0.0 0.05" rpy="0.0 0.0 0.0"/>
  </joint>

  <!-- 3. LINK DO CHASSI PRINCIPAL -->
  <link name="chassi_link">
    <inertial>
      <mass value="1.5"/>
      <origin xyz="0.0 0.0 0.0"/>
      <inertia ixx="0.005" ixy="0.0" ixz="0.0" iyy="0.005" iyz="0.0" iz
    </inertial>
    <collision>
      <geometry>
        <box size="0.3 0.2 0.1"/>
      </geometry>
    </collision>
    <visual>
      <geometry>
        <box size="0.3 0.2 0.1"/>
      </geometry>
    </visual>
  </link>

  <!-- 4. JUNTA CONTÍNUA PARA A RODA -->
```

```

<joint name="roda_esquerda_joint" type="continuous">
  <parent link="chassi_link"/>
  <child link="roda_esquerda_link"/>
  <origin xyz="0.0 0.12 -0.02" rpy="0.0 0.0 0.0"/>
  <axis xyz="0.0 1.0 0.0"/>
</joint>

<!-- 5. LINK DA RODA -->
<link name="roda_esquerda_link">
  <inertial>
    <mass value="0.2"/>
    <origin xyz="0.0 0.0 0.0"/>
    <inertia ixx="0.0001" ixy="0.0" ixz="0.0" iyy="0.0001" iyz="0.0"
  </inertial>
  <collision>
    <origin xyz="0.0 0.0 0.0" rpy="1.5708 0.0 0.0"/>
    <geometry>
      <cylinder radius="0.03" length="0.02"/>
    </geometry>
  </collision>
  <visual>
    <origin xyz="0.0 0.0 0.0" rpy="1.5708 0.0 0.0"/>
    <geometry>
      <cylinder radius="0.03" length="0.02"/>
    </geometry>
  </visual>
</link>

</robot>

```

3.0.1 Análise Linha por Linha do Código Exemplo

Para garantir o entendimento completo do código acima, vamos isolar o papel de cada bloco declarado:

- **Cabeçalho XML e tag <robot>:** A primeira linha indica que o arquivo segue o padrão XML. A tag <robot> delimita o escopo completo do nosso robô e define o seu nome oficial para o ROS 2 (robo_minimalista). O parâmetro xmlns:xacro estende o arquivo para aceitar macros futuramente.

- **A Origem no Solo (`base_footprint`):** O robô começa com um link vazio no chão. A junta `base_joint` é do tipo `fixed` (não se move) e serve apenas para dizer que o corpo real do robô (`chassi_link`) está flutuando a 0.05 metros (5 centímetros) de altura em relação ao solo (eixo Z).
- **O Bloco do Chassi (`chassi_link`):** É modelado como uma caixa (`box`) com dimensões de $30 \times 20 \times 10$ centímetros (`size="0.3 0.2 0.1"`). Note que o mesmo tamanho foi espelhado tanto no `<visual>` quanto no `<collision>`, garantindo que o chassi físico ocupe o mesmo espaço que os nossos olhos enxergam na simulação. Ele possui uma massa definida de 1.5 kg.
- **A Articulação da Roda (`roda_esquerda_joint`):** Conecta o chassi à roda. Como rodas giram livremente, o tipo escolhido foi `continuous`.
 - A tag `origin` desloca a roda 12 centímetros para a esquerda (eixo $Y = 0.12$) e 2 centímetros para baixo (eixo $Z = -0.02$) em relação ao centro do chassi.
 - A tag `axis xyz="0 1 0"` é crucial: ela informa ao motor de física que esta roda só pode girar em torno do seu eixo Y local (eixo longitudinal do rolamento).
- **O Bloco da Roda (`roda_esquerda_link`):** É modelado geometricamente como um cilindro (`cylinder`) com raio de 3 cm e largura de 2 cm.
 - *Detalhe importante de rotação:* Por padrão, o URDF desenha cilindros em pé (apontados para o eixo Z). Como rodas precisam ficar deitadas para rodar no chão, aplicou-se uma rotação de 1.5708 radianos (90°) no parâmetro `rpy` (*Roll*) dentro dos blocos `visual` e de colisão para deitá-lo adequadamente no eixo X .

4 AUTOMATIZAÇÃO DO AMBIENTE COM LAUNCH FILES

No ROS 2, para colocar o robô em operação, não basta apenas ter os arquivos de modelagem estrutural prontos. É necessário executar múltiplos nós de software em paralelo: um nó para ler e publicar o URDF, outro para gerenciar os estados das articulações e, finalmente, as interfaces gráficas de simulação e visualização. Executar cada um desses nós manualmente em terminais separados torna-se inviável.

Para solucionar isso, utilizamos os **Launch Files** escritas em Python. Eles funcionam como scripts de orquestração que configuram parâmetros, resolvem caminhos de pacotes e inicializam todo o ecossistema do robô com um único comando no terminal.

4.0.1 O Papel dos Nós Fundamentais de Inicialização

Antes de analisar o código de inicialização, é crucial entender os dois nós nativos do ROS 2 que dão suporte à visualização do robô:

- **robot_state_publisher**: Este nó é o coração da cinemática no ROS 2. Ele recebe o arquivo URDF completo (ou o XACRO processado) como um parâmetro de texto. Sua função é calcular continuamente as transformações espaciais tridimensionais (as coordenadas de translação e rotação) entre todos os *links* do robô. Ele publica esses dados no sistema de transformações globais do ROS 2, conhecido como **TF** (*Transform Library*). Sem este nó, o RViz2 não saberia onde o braço ou as rodas estão em relação ao chassi.
 - **joint_state_publisher_gui**: Quando estamos testando o robô apenas no RViz2 (sem um simulador físico como o Gazebo ativo), as juntas móveis do tipo `revolute` ou `continuous` não possuem motores virtuais enviando dados. Este nó cria automaticamente uma interface gráfica simples com barras deslizantes (*sliders*). Ao mover essas barras na tela, o nó publica dados simulados de ângulo no tópico `/joint_states`, permitindo que o desenvolvedor teste se as rodas e articulações estão girando no eixo correto antes de partir para a simulação física.
-

4.0.2 Exemplo Prático: Script de Inicialização (`display.launch.py`)

Crie um arquivo chamado `display.launch.py` dentro da pasta `launch/` do seu pacote. Este script processará o arquivo XACRO estrutural do robô BART, inicializará os publicadores de estado e abrirá o RViz2 de forma integrada:

```
import os
```

```

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
import launch

def generate_launch_description():

    # 1. Definição do nome do pacote e caminhos locais
    package_name = 'nome do pacote'
    pkg_share = get_package_share_directory(package_name)

    # Caminho para o arquivo principal URDF.XACRO do robô
    xacro_file = os.path.join(pkg_share, 'urdf', 'arquivo.urdf.xacro')

    # Processamento do arquivo XACRO via comando do sistema
    # (Gera a string XML limpa com as macros resolvidas)
    robot_description_raw = launch.substitutions.Command(['xacro ', xacro_file])

    # 2. Configuração e inclusão do Launch nativo do Gazebo
    # Localiza o pacote padrão do gazebo_ros para carregar o simulador
    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')

    # Inclui o script de inicialização padrão do Gazebo (abre um mundo)
    launch_gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_gazebo_ros, 'launch', 'gazebo.launch.py')
        )
    )

    # 3. Declaração dos Nós do ROS 2 para execução

    # Nó do Robot State Publisher (Obrigatório para o Gazebo entender a posição)
    node_robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        output='screen',
        parameters=[{

```

```

        'robot_description': robot_description_raw,
        'use_sim_time': True # Alinha o tempo do nó com o relógio d
    }]
)

# Nó de Spawn: Injeta a entidade do robô dentro do Gazebo
# Ele escuta o parâmetro 'robot_description' publicado e materializ
node_spawn_entity = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=[
        '-topic', 'robot_description', # Tópico de onde ler o URDF
        '-entity', 'nome_do_robo',      # Nome do robô dentr
        '-x', '0.0',                      # Posição X inicial de spawn
        '-y', '0.0',                      # Posição Y inicial de spawn
        '-z', '0.05'                      # Posição Z inicial (leveme
    ],
    output='screen'
)

# 4. Retorno da Descrição de Inicialização contendo o Gazebo e os N
return LaunchDescription([
    launch_gazebo,
    node_robot_state_publisher,
    node_spawn_entity
])

```

4.0.3 Análise do Processo de Injeção Física

- **A importância do `use_sim_time`:** Ao rodar robôs em simuladores físicos, o relógio do computador real (tempo de CPU) e o relógio da simulação física podem sofrer pequenos desvios. Definir `'use_sim_time': True` força o nó `robot_state_publisher` a sincronizar os cálculos matemáticos das juntas com o tempo interno do Gazebo. Se isso for omitido, as transformações de TF ficarão defasadas, quebrando os pipelines de SLAM e navegação.
- **O script `spawn_entity.py`:** Este utilitário fornecido pela comunidade no pacote `gazebo_ros` elimina a necessidade de converter manualmente os arquivos XACRO para arquivos SDF nativos do Gazebo. O argumento `-topic robot_description` faz com que ele capture diretamente a string XML que o `robot_state_publisher` já

processou e colocou na memória do ROS 2, injetando o robô de forma limpa e em tempo de execução.

- **A coordenada Z positiva ($-z = 0.05$):** É uma boa prática de engenharia de simulação realizar o *spawn* do robô ligeiramente acima do solo (5 centímetros no exemplo). Isso evita que as rodas colidam de forma violenta com a malha geométrica do chão no exato milissegundo em que o motor de física é inicializado, o que poderia fazer o robô ser arremessado para o alto devido a instabilidades numéricas de contato inicial.

5 ADEQUAÇÃO PARA O ISAAC SIM

5.0.1 Como converter arquivo *xacro* em *urdfs*

Com o arquivo URDF finalizado e sua funcionalidade devidamente verificada por meio do arquivo *launch* no ROS 2, o próximo passo consiste em prepará-lo para o ambiente do NVIDIA Isaac Sim. Como o simulador requer um arquivo URDF estático e completamente resolvido para realizar a importação, é necessário processar as macros do Xacro.

Para isso, navegue até o diretório onde o arquivo `seu_modelo.urdf.xacro` está localizado, abra o terminal e execute o seguinte comando:

```
xacro seu_modelo.urdf.xacro > seu_modelo.urdf
```

Esse comando processa todas as propriedades e macros do Xacro, gerando um arquivo URDF limpo, completo e pronto para ser importado diretamente no Isaac Sim utilizando a ferramenta *URDF Importer*.